

UTTEC 교육 · 원리 이해 자료 (초보자용)

"온도 높으면 빨간불 켜줘" 한마디가 진짜 회로를 움직이기까지 — Claude가 회로를 읽고·핀을 배정하고·실행하는 원리

코딩도 전자회로도 몰라도 괜찮습니다. 왜 **평범한 한국어 한 문장**이 눈앞의 LED를 켜고 부저를 울리는지, 그 **보이지 않는 과정**을 처음부터 끝까지 그림과 함께 풀어 드립니다.

대상 · 완전 초보자

키트 · UTTEC Shield (라즈베리파이)

포함 · 실제 회로도 · 핀 배정표 · 코드

 "온도가 35도 넘으면 빨간불 켜고 부저 울려줘"



 **빨간 LED 점등** +  **부저 뻐-**

사람이 핀 번호도, 파이썬 문법도 몰랐는데 실제 장치가 반응합니다. 마법처럼 보이지만, 뒤에는 **명확한 원리**가 있습니다. 이 문서가 그 원리입니다.



OVERVIEW · 이 문서가 답하는 3가지

이 문서가 풀어줄 세 가지 질문

1

Claude는 회로를 어떻게 아나?

사람이 안 알려줘도 어떤 부품이 어디 붙었는지 아는 비결

2

어떻게 올바른 핀을 고르나?

"빨간불"이라는 말 → 정확히 GPIO17을 만지는 이유

3

자연어가 어떻게 프로그램이 되나?

한국어 주문 → 파이썬 코드 → 실제 동작까지

핵심 한 줄. 비밀은 회로가 미리 "정해져" 있고, 그 내용이 컴퓨터가 읽을 수 있는 "글"로 적혀 있으며, Claude가 그 글을 자동으로 읽기 때문입니다. 아래에서 하나씩 봅니다.

먼저, 무대 위 네 주인공



나 (주문자)

한국어로 "이렇게 해줘" 하고

주문

만 합니다.



Claude (번역가·작성자)

내 말을 알아듣고

올바른 코드로 번역

해 만들어 줍니다.



라즈베리파이 (실행자)

그 코드를

실제로 실행

해 전기 신호를 내보냅니다.



UTTEC 키트 (무대·회로)

LED·센서·부저가

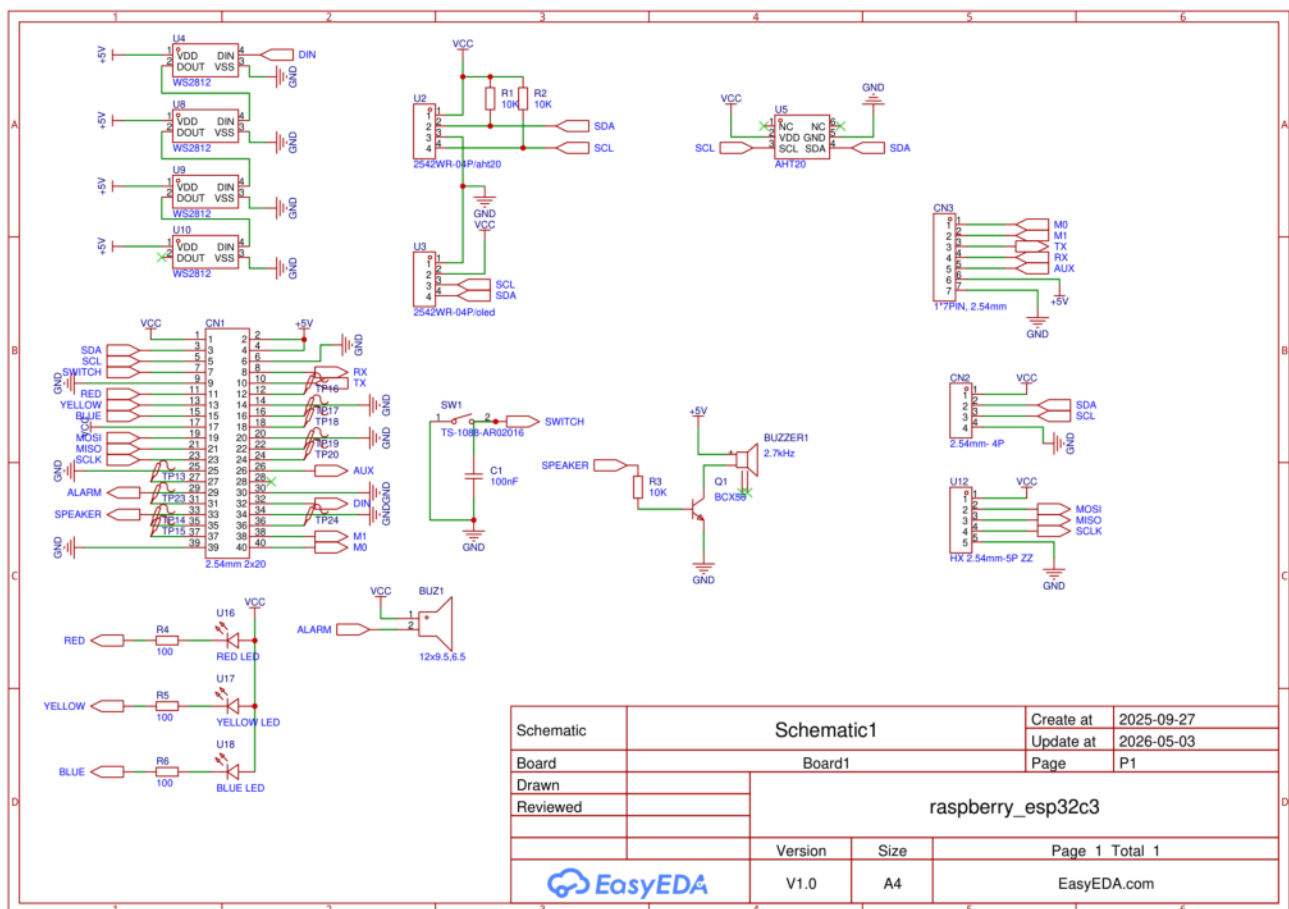
정해진 핀

에 붙어 있는 실습 보드.

이 문서의 관심사는 가운데 두 명 사이 — **내 말**이 **Claude**를 거쳐 어떻게 **라즈베리파이**의 실제 동작으로 바뀌는가입니다.

비밀 ①: 회로는 이미 "정해져" 있다

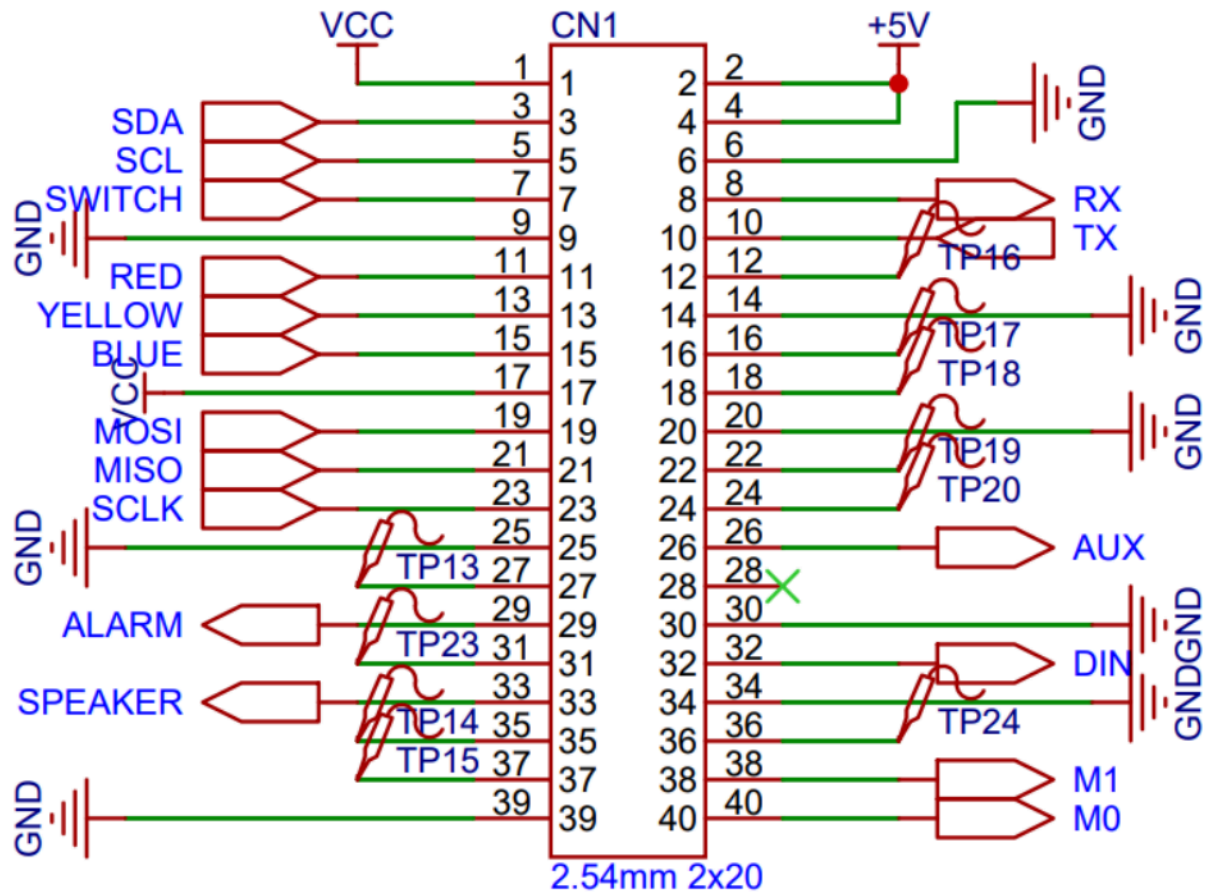
아무 부품이나 아무 데나 꽂는 게 아닙니다. UTTEC 키트(UTTEC Shield)는 **표준 설계**라서, 같은 부품은 언제나 같은 핀에 연결돼 있습니다. 그래서 "이 키트에서 빨간 LED는 항상 GPIO17"처럼 **약속이 고정**돼 있습니다.



▲ UTTEC Shield 전체 회로도 (설계 원본, EasyEDA · V1.0 · 2026-05-03)

복잡해 보여도 겁먹을 필요 없습니다. 이건 "어떤 부품이 라즈베리파이의 어느 다리(핀)에 연결됐는지"를 그린 **설계 지도**입니다. WS2812(네오픽셀)·AHT20(온습도)·OLED(화면)·LED 3색·부저·스피커·스위치·LoRa(무선) 모듈이 모두 여기 그려져 있습니다. 이 그림이 모든 것의 출발점입니다.

위 회로도에서 **딱 한 부분**, 라즈베리파이와 연결되는 **40핀 커넥터(CN1)**만 크게 보면 이렇게 정리됩니다 — 어떤 신호가 몇 번 핀인지를 한눈에 보여주는 **핀 배정 지도**입니다.



▲ 40핀 커넥터(CN1) 신호 배정 지도 — 왼쪽에 SDA·SCL(온습도/화면), RED·YELLOW·BLUE(LED 3색), ALARM(부저)·SPEAKER(스피커), 오른쪽에 DIN(네오픽셀)·RX·TX(무선)·M0·M1·AUX(무선 제어) 등 부품 이름이 곧 핀 위치로 적혀 있습니다.

왜 이게 중요한가. 회로가 매번 똑같이 정해져 있으니, "이 키트에서 빨간불 = GPIO17"이라는 사실을 한 번 적어두면 영원히 재사용할 수 있습니다. 이게 자연어 주문이 가능한 첫 번째 토대입니다.

비밀 ②: 그 회로를 "컴퓨터가 읽는 글"로 적어 둔다 — port_map.py

사람은 위 그림을 보고 이해하지만, Claude(컴퓨터)는 글(텍스트)을 읽습니다. 그래서 회로도의 내용을 똑같이, 글자로 옮겨 적은 파일이 있습니다. 바로 reference/하드웨어/port_map.py 입니다. 그림과 글은 완전히 같은 내용입니다.

```
reference/하드웨어/port_map.py (핵심 발췌)

# UTTEC Shield 보드 포트 맵 - 회로도(ai_smart_factory_schematic.pdf)를 글로 옮긴 것

# I2C (온습도 AHT20 · 화면 OLED)
PIN_SDA    = 2    # GPIO2   (Pin 3)
PIN_SCL    = 3    # GPIO3   (Pin 5)

# LED 신호등 3색
PIN_RED     = 17   # GPIO17  (Pin 11) - 빨강
PIN_YELLOW  = 27   # GPIO27  (Pin 13) - 노랑
PIN_BLUE    = 22   # GPIO22  (Pin 15) - 파랑

# 네오픽셀 · 부저 · 스피커 · 스위치
PIN_DIN     = 12   # GPIO12  (Pin 32) - WS2812 4개
PIN_ALARM   = 5    # GPIO5   (Pin 29) - 부저
PIN_SPEAKER = 13   # GPIO13  (Pin 33) - 스피커
PIN_SWITCH  = 4    # GPIO4   (Pin 7)  - 탭트 스위치

# 무선(LoRa E22) - UART + 모드제어
PIN_TX = 14; PIN_RX = 15      # GPIO14/15 (Pin 8/10)
PIN_M0 = 21; PIN_M1 = 20; PIN_AUX = 7 # GPIO21/20/7 (Pin 40/38/26)
```

같은 내용을 표로 보면 이렇습니다. 부품 이름 ↔ GPIO 번호 ↔ 실제 핀 번호가 1:1로 묶여 있습니다.

부품 (내가 말하는 이름)	GPIO 번호	물리 핀	비고
온습도 AHT20 · 화면 OLED	GPIO2 / 3	3 / 5	I2C 통신
● 빨강 LED	GPIO17	11	신호등 3색
● 노랑 LED	GPIO27	13	

 파랑 LED	GPI022	15	
 네오픽셀 WS2812 ×4	GPI012	32	DIN
 부저	GPI05	29	경보음
 스피커	GPI013	33	멜로디(PWM)
 스위치	GPI04	7	버튼 입력
 무선 LoRa (UART)	GPI014 / 15	8 / 10	TX / RX
 무선 LoRa (모드제어)	GPI021 / 20 / 7	40 / 38 / 26	M0 / M1 / AUX

정리. 회로도(그림) = port_map.py(글) = 위 표 — 셋은 표현만 다를 뿐 똑같은 약속입니다. Claude는 이 중 글(port_map.py)을 읽어 회로를 "이해"합니다.

비밀 ③: Claude가 그 지식을 "자동으로" 갖는 법 — CLAUDE.md

아무리 좋은 핀맵이 있어도, Claude가 **안 읽으면** 소용없습니다. 그래서 **Claude Code**가 시작될 때 **자동으로 읽는 파일**에 기본 지식을 심어 둡니다. 그 파일이 wiki 폴더의 CLAUDE.md 입니다.

1

claude 실행

wiki 폴더에서 claude 입력

2

CLAUDE.md 자동 읽기

커리큘럼·폴더구조·핀맵은
reference/하드웨어" 라고 적혀 있음

3

필요 시 port_map 읽기

부품을 다룰 땐 핀맵 파일을 펼쳐 정
확한 번호 확인

핵심 설계. 항상 알아야 할 기본 지식(커리큘럼·핀맵 위치)은 CLAUDE.md에 심고, **자세한 핀 번호**는 reference/하드웨어/port_map.py가 단일 출처입니다. 그래서 여러분이 **매번 회로를 설명하지 않아도** Claude가 이미 알고 시작합니다.

※ 반대로 말하면 — **핀맵에 없는 외부 부품**(직접 사온 새 센서 등)은 Claude가 모릅니다. 그럴 땐 여러분이 "이건 GPIO○○에 연결했어"라고 **알려줘야** 합니다. (문서 끝 "한계" 참고)

한 편의 흐름으로 보기 — 실제 예시

이제 세 비밀을 합쳐, 주문 한 문장이 실제 동작이 되는 전 과정을 따라가 봅니다.

● ● ● 내가 Claude에게 한 주문 (자연어)

온도가 35도를 넘으면 빨간 LED를 켜고 부저를 울려줘.

1

알아듣기

"온도"=AHT20 센서, "빨간 LED", "부저" 라고 파악

2

핀 배정

port_map에서 찾을 → 빨강=GPIO17, 부저=GPIO5, 온도=I2C(GPIO2/3)

3

코드 작성

그 핀 번호로 정확한 파이썬 코드를 만든다

4

실행·반응

라즈베리파이가 실행 → 눈앞 LED·부저가 동작

② 핀 배정 — Claude가 속으로 정리한 표

내 말	부품	Claude가 고른 핀
"온도"	AHT20 온습도 센서	I2C = GPIO2/3
"빨간 LED"	빨강 LED	GPIO17
"부저"	알람 부저	GPIO5

③ 코드 작성 — 핀 번호가 port_map과 정확히 일치

● ● ● Claude가 자동으로 만든 파이썬 코드 (요약)

```
import board, busio, adafruit_ahtx0
from gpiozero import LED, Buzzer

led_red = LED(17, active_high=False) # ← port_map PIN_RED
buzzer = Buzzer(5) # ← port_map PIN_ALARM
aht = adafruit_ahtx0.AHTx0(busio.I2C(board.SCL, board.SDA)) # ← GPIO2/3
```

```
t = aht.temperature
if t > 35:
    led_red.on(); buzzer.on()      # 위험 → 빨간불 + 경보
else:
    led_red.off(); buzzer.off()
```

여기가 마법의 정체입니다. 여러분은 "17"이나 "5" 같은 숫자를 **한 번도 말하지 않았지만**, Claude가 port_map을 읽고 **정확한 핀 번호를 스스로 채워 넣었습니다**. (심지어 빨강 LED가 active_high=False로 동작한다는 세부까지 회로 지식에서 반영합니다.)

④ 실행 → 눈앞의 반응

라즈베리파이가 이 코드를 실행하면, 센서 온도를 읽어 35도를 넘는 순간 **GPIO17에 전기가 흘러 빨간 LED가 켜지고, GPIO5로 부저가 울립니다**. 내가 한국어로 말한 그대로, 현실의 장치가 반응합니다.

왜 이렇게 될 수 있었나 — 네 가지 준비

이 편리함은 저절로 생긴 게 아니라, UTTEC가 네 단계로 미리 깔아 둔 판 덕분입니다.

1

표준화

키트를

자체 설계

로 고정 → 부품이 항상

같은 편

2

문서화

그 배선을

회로도 +
port_map.py

로 기록 → 기계가 읽을

지식

3

자동 로드

기본 지식을

CLAUDE.md

에 심음 → Claude가 자동

인지

4

도구화

검사 스킴·헬퍼 모듈로

더 쉽게

(예: /hardwareTest)

단계

없었다면?

있어서

표준화

키트마다 배선이 달라 매번 새로 설명

"이 키트의 빨간불=17"이 항상 참

문서화

사람 머릿속에만 있어 Claude가 못 읽음

글로 적혀 Claude가 읽고 코드에 반영

자동 로드

매번 회로를 붙여넣어야 함

실행하자마자 이미 알고 시작

도구화

오류가 나도 원인 파악이 어려움

스킬로 부품별 점검·확인

한 문장 요약. "똑같이 만들고(표준화) → 글로 적고(문서화) → 자동으로 읽히게 한다(자동 로드)." 이 세 가지가 비전공자도 자연어로 회로를 다루게 만든 핵심입니다.

직접 주문해 보기

접속 후(→ 「노트북 VSCode 접속 설명서」 참고) wiki 폴더에서 claude 를 실행하고, 아래처럼 **편한 한국어**로 말해 보세요. 핀 번호는 몰라도 됩니다.

- ▶ 쉬움 — "파란 LED를 3번 깜빡여줘."
- ▶ 쉬움 — "지금 온도랑 습도를 화면(OLED)에 보여줘."
- ▶ 보통 — "온도가 30도 넘으면 노란불, 35도 넘으면 빨간불에 부저까지 울려줘."
- ▶ 도전 — "버튼을 누를 때마다 네오픽셀 색을 빨강→초록→파랑으로 바꿔줘."

팁. 원하는 대로 안 나오면, **결과를 그대로 말해주고 고쳐 달라**고 하면 됩니다. 예: "부저가 계속 울려. 35도 넘을 때만 잠깐 울리게 바꿔줘."



마법에도 한계가 있습니다 (안전하게 쓰기)

- ▶ **핀맵에 없는 부품은 모릅니다.** 직접 사온 새 센서를 꽂았다면, "이건 GPIO○○에 연결했어"라고 알려줘야 합니다.
- ▶ **Claude도 틀릴 수 있습니다.** 만든 코드가 항상 완벽하진 않으니, **동작을 눈으로 확인**하고 이상하면 다시 요청하세요.
- ▶ **회로의 진실은 하드웨어 문서입니다.** 핀·회로가 헛갈리면 최종 기준은 `reference/하드웨어/(회로도·port_map)`와 UTTEC 하드웨어 담당입니다.
- ▶ **전원·결선은 조심.** +5V와 GND를 잘못 이으면 부품이 상할 수 있습니다. 배선 변경은 강사와 함께.

그래도 남는 진짜 가치. 여러분은 "무엇을 하고 싶은지"에 집중하고, "어떻게(핀·문법)"는 Claude가 채웁니다. 이게 이 교육이 비전공자에게 열어주는 새로운 방식입니다.

UTTEC 교육 · Claude가 회로를 읽고 명령을 실행하는 원리 (초보자용) | 회로 자료 출처 [wiki/reference/하드웨어](#) (회로도 · 40핀 배정 · port_map.py) · 함께 보기 「[노트북 VSCode wiki 접속](#) · [Claude 실행 설치 설명서](#)」
핀 번호를 외울 필요는 없습니다. 원리만 이해하면 충분합니다 — 나머지는 Claude가 합니다.